

A Machine Learning Approach to Examine the Reactionary Effects of Medical Testing on Rodents: A Comparative Analysis

1st Arham Siddiqui
Santa Clara High School
Santa Clara, California
arham.siddiqui61@gmail.com

2nd Connor Houle
Santa Clara High School
Santa Clara, California
cphoule@yahoo.com

3rd Daniel Ticau
Santa Clara High School
Santa Clara, California
danitica@gmail.com

Abstract—This research explores the reactionary effect of rodents to medical testing. The purpose of this project is to assist scientists and pharmaceuticals with techniques that can improve the reliability of drug discovery and aid companies in the development process of drugs and vaccines by proving the efficiency of supervised machine-learning models in such a setting. 13 features representing rodent reactions to drugs were artificially created and varied to simulate the results of the specimen observation period conducted by scientists in a range of 72 hours post-injection. We created two models, an SVC () and KNN (), our KNN having a 40% higher accuracy score on average, ultimately having 100% accuracy. After training multiple models with different hyperparameters, our algorithm is capable of pairing a new set of post-injection observations with a previously tested drug with 100% accuracy. We noticed significant differences in accuracy between the SVC and KNN models, with our KNNs having a 40% higher accuracy score on average. Additionally, we increased the size of our dataset from 13x1800 to 13x6000 to evaluate the models on a larger dataset. When using the Manhattan Distance Formula and the Euclidean Distance Formula, we noticed that Manhattan was the optimal formula for our algorithm. By identifying an effective drug-matching model coupled with supplemental research exploring its advantages, drawbacks, and optimal conditions, scientists and companies can use it as a reference point to further direct their development efforts. Such assistance to the development process will consequently reduce the time required, thus eliminating increased costs and delays in treatment for patients.

I. INTRODUCTION

The development of medical drugs is a complex and time-consuming process that typically requires 5-10 years, if not more, for most professional cases. This process is a multi-faceted undertaking requiring multiple stages, such as the conduction of lengthy laboratory testing to assess the safety, effectiveness, and efficiency of the product. Additionally, scientists must conduct extensive testing on animals before finally, being approved for human testing where scientists must conduct detailed experiments for 3 different phases. This tedious process consequently poses significant challenges for the medical industry as well as broad implications for society.

II. BACKGROUND RESEARCH

Such problems include the increased cost associated with the development process, delay in treatment, competitive prod-

ucts, and risk of failure. The median research and development investment for a new drug to be brought to market was estimated to be \$985.3 million [4], a considerable sum that is further increased depending upon the length of the development process. This cost is translated over-the-counter to the consumer, which can unfortunately make certain medications unaffordable for some. This exorbitant price tag leads to financial strain, overall distrust in the medical system, and hinders the use of necessary medications resulting in serious consequences for individuals' health and well being. On the topic of the health and well being of the patient, the length of a medications' development process can yield obvious consequences to those waiting for treatment, as they are left to suffer the symptoms of illness.

More specifically, for those with serious or life-threatening conditions, the prolonged roll-out of such medications is simply beyond frustrating [9]. A time-intensive production process also induces the competitive nature of the medical industry. With 9 of the top 10 pharmaceutical companies based in the US and most being active in drug production, research, and development, it goes without saying that the US medical industry brews aggressive competition [10]. Such an environment urges companies to use a more shortened, yet efficient, development process to break from competitors and earn a spot on the shelf.

Drug production, no matter the scale, will likely result in failure in some form [5]. Such failure prompts reiteration and improvement in the overall product. Failure during a drawn-out production process can cause major setbacks for the company and patients waiting for the treatment, as the time and money spent can never be replaced. Therefore, it is ideal that such flaws are brought to light early in the process, or as soon as possible. With a more efficient development process, failure points can be exposed quicker, and improvements can be applied to proceed with the process in a timely manner.

This evidence points to a quicker development process being desirable for medical companies seeking the optimal cost, efficiency, and competitive nature of their drugs. Machine learning, the sub-field of artificial intelligence involving the training of algorithms to make decisions based on patterns

in large amounts of data, can be applied to enhance the drug development process in multiple contexts. This research particularly explores the use of machine learning to help identify matching drugs based on the reactionary effects of medical testing on rodents. More specifically, we conduct a comparative analysis between various classifiers, model architectures, as well as data-set manipulation to maximize the efficiency of a supervised multi-class classification model in this setting.

III. RELATED WORK

Our goal was not to create an entirely new field but to instead revolutionize the niche in which we worked.

Other efforts have focused on similar niches such as FIT [11], a team focusing on the translatory effects within genes from mice to the equivalent human phenotypes, which helps scientists predict the genotype translations between the two species without having to use live test subjects as “guinea pigs”. Similarly, they created a machine learning model to assist scientists in their research by helping them predict the expected changes between the species.

Conversely, our model’s goal is to help compare the reactionary effects of pre-approved drugs to a drug in the development process. Allowing scientists to better understand how their drug may differ from widely accepted and used drugs, hopefully speeding up the FDA preparation and approval processes. Additionally, we hope scientists can use our program to promote their drug’s benefits against the competition.

Outreach

To deepen our understanding of the topic at hand and gain a more credible perspective on how we may be able to solve it, we contacted two scientists currently working in similar fields.

The first has a Ph.D. in biochemistry and currently tests pharmaceutical drugs on mice in labs.

The second is currently attending MIT and participating in daily work at the Sinclair lab, which does research regarding the aging of humans and how to reverse it. They use rodents for a majority of their tests and research.

They gave us insight into the applicability of our features, and how they would test in the real world. They also gave us examples of which features would relate to a rodent’s biological system in response to a drug, and which ones wouldn’t (eg. dopamine, epinephrine, cholesterol, and glucose levels).

IV. ENGINEERING GOAL

To assist scientists in their research and their work toward human testing approval we created Neuroratomic ML: a machine learning approach to examine the reactionary effects of medical testing on rodents: a comparative analysis.

The purpose of this research is to aid scientists and pharmaceutical companies in the development process of drugs and vaccines by proving the efficiency of supervised machine-learning models in such a setting.

By identifying an effective drug-matching model coupled with supplemental research exploring its advantages, drawbacks, and optimal conditions, scientists and companies can use it as a reference point to further direct their development efforts. Such assistance to the development process will consequently reduce the time required, thus eliminating increased costs and delays in treatment for patients.

Additionally, when comparing their own drug to existing products on the market, companies can promote their product by giving real numerical data regarding ways theirs is better.

This research will also relay a thorough analysis of the development process for this model in order to further the understanding of supervised multi-class classification models among those interested.

V. CRITERIA AND CONSTRAINTS

Our model must be able to accurately match the reactionary effects of a given drug to that of the closest preexisting approved drug from one of our five drugs, or classes, with an accuracy of at least 90% (percent of total test data identified correctly).

This accuracy will be calculated for each class by using the number of true positives and true negatives divided by the number of total classifications. The true positives and true negatives are the number of correct classifications within a given class, whereas the remaining values (false negatives and false positives) consist of the incorrect classifications within that class.

Originally, we planned on adding time constraints to prove our program’s efficiency, but figured that in a real-world setting as long as our time to run was under 30 minutes, (which it was), time wouldn’t be a practical issue to solve.

Metrics such as precision, recall, and f1-score are also analyzed. In regards to these metrics, precedence will be placed on precision, as minimizing false positive matches is of particular importance in such a sophisticated setting where a mismatched drug could monumentally impair the course of production.

VI. LIBRARIES/BASIC SOFTWARE

We decided to use Google software to run and test our code as well as store our datasets because of Google’s ease of use and quick connection between applications. We used Google Colab, Google Sheets, and Google Docs.

In order to accomplish our goal, we used many different python libraries and functions to allow us to use machine learning algorithms/models. These include SVC OVO, SVC OVR, KNN, etc. These were all imported from the “sklearn” library, a common machine-learning library used in many python scripts.

VII. FEATURES

To simulate a real-world applicable machine learning model, we handpicked features which, once observed, would give scientists better insight into the translation of reactions from rodent testing to human applicants.

We picked these variables based on the internal effects of existing drugs and drug tests, as well as input from a lab-testing pharmaceutical scientist.

Our included features include blood pressure, heartbeat/min, average weight gain/loss over 72 hrs, body temperature, cholesterol levels, glomerular filtration rates, glucose levels, total white blood cell count, % of neutrophils in the blood, % of lymphocytes in the blood, % of monocytes in the blood, % of eosinophils in the blood, and the % of basophils in the blood.

The details of each feature are expanded upon in our Feature Explanation Document [see Appendix A].

VIII. DATASET

The dataset used is synthetically created and composed of 13 numerical features mentioned previously. These values are based on true and thoroughly observed baseline accepted values as explained in the Feature Explanation Document [see Appendix A].

Each feature is further broken into 3 subcategories: average, high, and low. These benchmarks are pointers for the different possible ranges of the respective feature. The high and low ranges are derived from the axioms defined in the explanation document, and are consistent with real-life principles. In perspective, this paper and the research conducted is all meant to simulate real world possibility. Since this dataset does not include any industry-specific drugs, the differentiating feature ranges are applied to model real-world variation observed in reactionary effects. We synthesized 6 artificial drugs labeled Drug A - F. Each drug contains all 13 features with different variations of said ranges. These variations can be seen in the Drug Description Document [see Appendix B]. These variations were applied to our dataset by using this equation, where R is a random decimal value between 0 and 1, u is the upper limit to our range, and l is the lower limit of our range, to create said random values:

$$R * (u - l) + l \quad (1)$$

The category listed on each drug is simply the corresponding letter in its name. This feature has been implemented to allow for the training and testing of our data in the methods section of this paper.

The breakdown for this dataset is as follows. Each drug contains 13 features, each of which belonging to one of the 3 different ranges, and there are 300 samples of each feature. Using the “concat()” function from the pandas library, stacking all drug datasets together creates a matrix sized 1800 x 14.

To process this dataset into a working environment, we start by reading in each drug individually and then stack all drugs into a single dataframe. After doing so, all duplicate values are checked and eliminated to prevent incorrect or misleading predictions based on overfitting. Duplicate identification was done through the “duplicated()” pandas function. Elimination was done through the “drop_duplicates()” pandas function.

For this dataset, we used an 80/20 training and testing set split. This was done with the use of the “train_test_split()”

scikit-learn function. This leaves a training feature matrix sized 1440 x 13, testing features 360 x 13, training labels 1440 x 1, and testing labels 360 x 1.

Throughout the course of this research, this dataset is altered to analyze performance results after alterations. Feature extraction and other forms of dataset manipulation will be expanded upon in the methods section of this paper.

IX. METHODS

SVM

We started by using an SVM, or support vector machine. This is a simple machine-learning algorithm that is used for baseline training. The goal of this model, more specifically an SVC (support vector classifier) shown in Figure 1, is to create a maximum margin classifier based on the data given, which is a hyperplane that maximally separates two categories of data, to provide a classification when given new data.

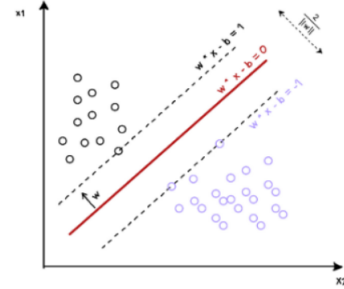


Fig. 1.

Since the problem of this research pertains to multi-class classification, using the rudimentary SVC model will not work, as they are intrinsically meant for binary classification. There are multiple ways to tackle this problem. More specifically, OVO (one vs one) and OVA/OVR (one vs all/rest), are two of the most commonly used approaches. OVO, or the one vs one approach, creates a binary classifier for each pair of classes. For example, a 3-class classification problem would require 6 classifiers as shown in Figure 2.

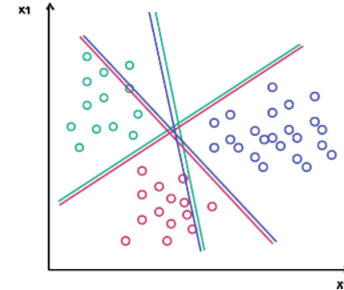


Fig. 2.

The OVA/OVR, or one vs all/rest approach, is executed exactly as the name suggests. A hyperplane is derived for each class to separate it from all the others combined. For this

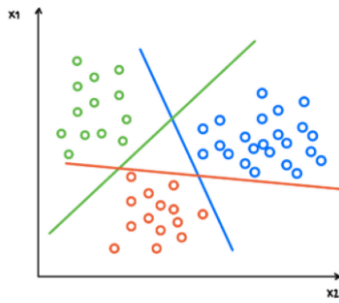


Fig. 3.

approach, the same 3-class classification problem will require only 3 classifiers, which can be seen in Figure 3.

After an understanding of both of these methods, we decided to run our model using the OVO approach first. To do so, we implemented the Support Vector Classifier as SVC from the Scikit-Learn Library. We set the “decision_function_shape” parameter to “ovo” to specify our approach. The next step was to fit the model with our previously split data. This was done through the “fit()” function from the Scikit-Learn Library. As we were training the model, our “X_train” and “y_train” matrices were passed in as arguments for the “fit” function. The last step was generating predictions on our testing set. To do so, we used the “predict()” function from the Scikit-Learn Library, passing in our allocated testing data: “X_test”.

After analyzing the results of the SVC (OVO) we decided to use the SVC with the OVR approach. To reiterate, the goal of the OVR model is to derive a hyperplane for each class to separate it from all the others combined, hence the name: one vs rest. We utilized the OneVsRestClassifier from the Scikit-Learn Multiclass Library to implement this model. We fit this model onto our previously split dataset. Just as we did with the previous model, we used the “predict()” function from the Scikit-Learn Library to make predictions on our “X_test” matrix. We stored these predictions in “y_pred”.

KNN

The next model used is a KNN, or K Nearest Neighbors model that doesn’t require training. Classification is achieved by first plotting all data samples onto a plane. When a test sample is introduced as a data point, the distance between it and all the other training samples is calculated using distance metrics, the most common being Euclidean and Manhattan distance. After this distance is calculated, the model selects K - the number of “nearest neighbors” - of the closest training samples to determine the prediction of the classification.

This is done through a simple majority-based voting algorithm. KNN is advantageous in this setting as it can differentiate well between classes that have stark differences between features, such as Drug A and Drug B, the former composed of all low values, whereas the latter being composed of all high values. This model, however, starts to falter in the classification between classes that have overlapping values, as certain data samples can be misidentified because of their

similar positioning on the plane. We applied this model to our dataset and altered the number of nearest neighbors along with the power parameter which controls the type of distance calculation used. A visual representation of this model can be viewed in Figure 4.

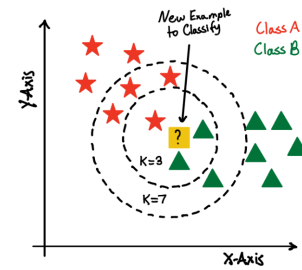


Fig. 4.

We set the two main parameters, n (amount of neighbors) and p (type of distance calculation) to the default value, which was 5 neighbors and the Euclidean Distance Formula. As done before, we fit this model on our training data from our split dataset. The model predictions were stored in “y_pred”, which were made from the “predict()” function from the Scikit-Learn Library.

After obtaining a significant improvement in the accuracy and precision scores of our model, we decided it would be best to capitalize on the performance of the KNN model. To do so efficiently, we decided to visualize all combinations of reasonable n-neighbor values with either Euclidean or Manhattan Distance. We created this python program to figure out the most optimal hyperparameters which ended up being 3 neighbors with the Manhattan Distance Formula. After applying these parameters to our model, we got a 98.33% accuracy score. This was extremely impressive, and upon increasing the size of our dataset, we achieved a 100% accuracy score.

A visualization of the hyperparameter accuracies can be seen in Figure 5.

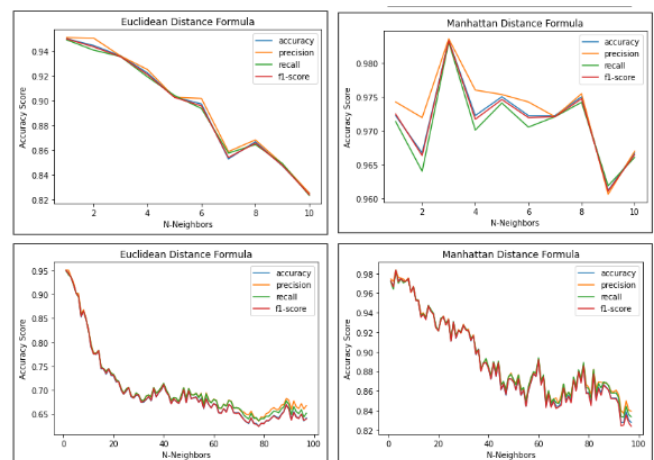


Fig. 5.

X. RESULTS & ANALYSIS

SVM Analysis

For the SVC OVO, once our predictions were stored in “y_pred”, we utilized the “accuracy_score()” function from the Scikit-Learn Metrics Library to evaluate the accuracy of our model. Specifically, we passed in “y_test” and “y_pred” as we were comparing the model’s predicted labels to the actual labels. The accuracy of this model was around 49.2%.

To get a better understanding of this accuracy, we decided to use a confusion matrix that visualizes the incorrect and correct classifications for each class. To plot said matrix, we used the “confusion_matrix()” function from the Scikit-Learn Metrics Library and passed in our “y_test” and “y_pred” matrices, for the same reasons as the accuracy score function.

To add a color gradient to our confusion matrix for visual appeal and trend detection, we utilized the “heatmap()” function from the Seaborn Library. After passing the confusion matrix into this function, we were able to display a colored version of the matrix as shown in Figure 6.

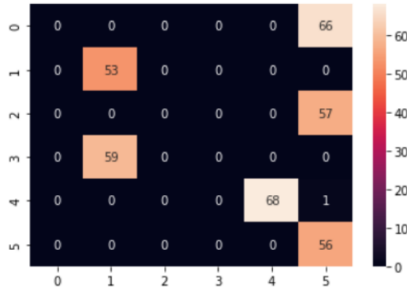


Fig. 6.

A model with 100% prediction accuracy will have a confusion matrix with a clear diagonal descending from left to right. This is true as all other values in the matrix will be zero and only the correct classifications will be colored in. An ideal confusion matrix can be seen in Figure 7.

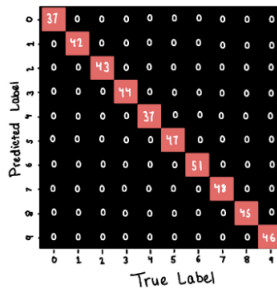


Fig. 7.

Comparing the matrix in Figure 6 to our matrix in Figure 7, we can locate the specific classes that our SVC (OVO) inaccurately predicted. Looking back on an ideal matrix, there should only be one colored box for each class, whereas our matrix has 2 colored boxes for class 1 and 3 colored boxes

for class 5. Analyzing this further, these boxes are not minor outliers but major clusters of classifications ranging from 57-66 incorrect classifications per misplaced prediction. Using the confusion matrix, we can ultimately determine that our SVC (OVO) commonly confused class 1 with class 3, and class 5 with classes 0 and 2. Converting these classes into our data set means that each drug is equivalent to its corresponding class index. This means that Drug A is Class 0, Drug B is Class 1, etc. Translating the results from our confusion matrix, we can infer that our SVC (OVO) commonly confused Drug B with Drug D, and Drug F with Drug A and Drug C. To gain insight into the model’s difficulties in distinguishing between these classes, we examined the Drug Description Document [see Appendix B].

We noticed that Drug B and Drug D were both frequently detected as Drug B, and never as D. We believe that this could have caused issues in our classification because of very similar ranges between features between these two drugs.

Examining Drug F in comparison to Drugs A and C, we observed that our model read all three drugs as Drug F, most likely due to very similar features and overlapping ranges. We inferred that this is a viable cause for incorrect predictions as this could very well happen in a real-world testing environment.

We utilized the OneVsRestClassifier from the Scikit-Learn Multiclass Library to implement this model. We fit this model onto our previously split dataset. Just as we did with the previous model, we used the “predict()” function from the Scikit-Learn Library to make predictions on our “X_test” matrix. We stored these predictions in “y_pred”.

We obtained the accuracy of the model by passing in our predicted and testing label data into the “accuracy_score()” function from the Scikit-Learn Metrics Library. The 65% accuracy of this model was significantly greater than that of the previous model. To further analyze this accuracy, we decided to plot a confusion matrix for the predictions of this model. Using the same method as the previous model we plotted a colored confusion matrix, which is shown in Figure 8.

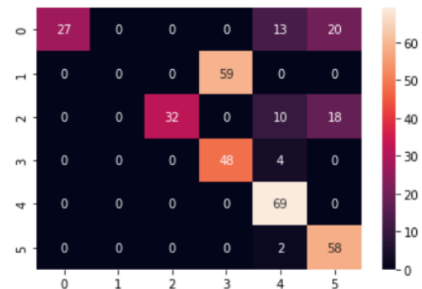


Fig. 8.

Once again, the classes numbered on the axes correspond to the index of the letter drug (Drug A = Class 0, Drug B = Class 1, etc). Visually observing this matrix, we notice that there are a lot more purple, pink, and magenta colors present as compared to the matrix of the SVC (OVO). The

legend on the right of our matrix indicates lower amounts of classifications per class. Additionally, there are many more inaccurate predictions as Drugs E and F are both misclassified as Drugs A and C. Something important to note is that Drug D is misclassified only as Drug B, which has an extremely high cluster of classifications (59) as opposed to all other misclassifications. Translating the results from our confusion matrix, we can infer that our SVC (OVR) occasionally confused Drugs E and F with Drugs A and C, while commonly confusing Drug D with Drug B. To gain insight into the model's difficulties in distinguishing between these classes, we once again examined the Drug Description Document [see Appendix B]. We noticed that Drugs E and F both have very similar values for body temperature, glomerular filtration rate, lymphocytes, monocytes, eosinophils, and basophils which could explain why both were misclassified into the same drug.

We believe that this could have caused issues in our classification because of the similarity between the two drugs' effects on the mice. We also believe that other models may be able to better identify the differences between the drugs, inherently improving our program's efficiency and accuracy.

Examining Drug D in comparison to Drug B, we observed that both caused numerous values to heighten abnormally. Some of the similar high values consisted of heart rate, body temperature, glucose levels, white blood cell count, neutrophils, and lymphocytes. We inferred that this is why these two drugs were most commonly confused. Although this may be true, we will have to deal with such an obstacle as it would most definitely appear in a real-world testing simulation.

Precision, Recall, and F1 are additional metrics to evaluate accuracy. In regards to these metrics, precedence will be placed on precision, as minimizing false positive matches is of particular importance in such a sophisticated setting where a mismatched drug could monumentally impair the course of production.

Using the "classification_report()" from the Scikit-Learn Metrics Library and passing "y_pred" and "y_test" (the model's prediction and actual label), the classification report for SVC (OVO) is shown in Figure 9.

	precision	recall	f1-score
a	0.00	0.00	0.00
b	0.47	1.00	0.64
c	0.00	0.00	0.00
d	0.00	0.00	0.00
e	1.00	0.99	0.99
f	0.31	1.00	0.47
accuracy			0.49
macro avg	0.30	0.50	0.35
weighted avg	0.31	0.49	0.36

Fig. 9.

Next, we performed this same procedure on the SVC (OVR) which can be seen in Figure 10.

Analyzing these metrics, we can see that OVR has a very high precision accuracy for drugs A and C. Additionally, it has

	precision	recall	f1-score
a	1.00	0.45	0.62
b	0.00	0.00	0.00
c	1.00	0.53	0.70
d	0.45	0.92	0.60
e	0.70	1.00	0.83
f	0.60	0.97	0.74
accuracy			0.65
macro avg	0.63	0.65	0.58
weighted avg	0.63	0.65	0.59

Fig. 10.

consistently good values of precision for the remaining drugs, scoring an average of 0.63. Within our testing, precision is very important as this value refers to the proportion of true positive predictions (the number of correct positive predictions) out of all positive predictions made by a model (true positive predictions plus false positive predictions). A high precision value indicates that a model is good at identifying true positive instances and has a low number of false positive predictions.

While OVR had an average precision rating of 0.63 or 63%, OVO had a rating of only 0.49 or 49%. OVO had perfect precision for only one drug, drug e, while it struggled greatly with drugs a, c, and d. This could be due to OVO only performing one-to-one comparisons, limiting the possibility to see other values and making connections and predictions between them. Conversely, OVR makes comparisons from one value to all of the other values, allowing it to make proper connections and predictions based on the entire data set at once. This limitation caused OVO to score an overall precision accuracy of .49 which is much too low for our needs.

In conclusion, we determined that for the SVC model, OVR is the preferred option. Not only was OVR's precision higher, but it also had better recall and f1 scores, making it the optimal choice for our testing.

KNN Analysis

Upon thorough analysis of the inaccuracies presented in the KNN (5 neighbors, Euclidean Distance Formula) confusion matrix shown in Figure 11, it is observed that Drugs A and C were commonly confused with each other, as well as Drugs B and D.



Fig. 11.

Viewing the Drug Description Document [see Appendix B], we noticed that the two drugs have very similar effects on the

mice in regards to heart rate, body temperature, glucose levels, white blood cell count, neutrophils, and lymphocytes. This may have been a reason why Drugs A and C were confused with each other.

Additionally, the similar heart rate, body temperature, glucose levels, white blood cell count, neutrophils, and lymphocyte values offered a reason as to why Drugs B and D may have been misclassified as one another. Using the same python script for our KNN Tuned 13x300 training run, but instead, with 1000 sets of training data, we found that we had tuned our model to perfection, yielding a 100% accurate result, as shown in Figure 12.

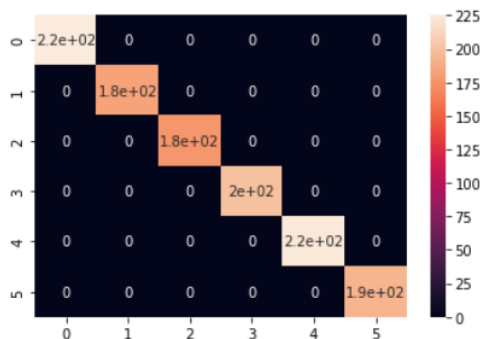


Fig. 12.

This is most likely because the extra training data allows the model to better define the features of each drug and create a figurative fine line between the classification of different drugs. As we had finally created a perfect model with 100% accuracy, we stopped searching for better models to utilize in our project here.

XI. CONCLUSION

After training and testing our model on simulated data based on realistic ranges, we concluded that our model stands as the skeletal structure for a potentially big help for drug testing scientists in the future.

We far surpassed our project goal of a 95% accuracy score on drug identification and ended up with a model that contained 100% accuracy.

We accomplished this goal by using a KNN model with 3 neighbors and the Manhattan Distance Formula for neighbor detection on a dataset composed of 1000 data samples per drug, with 6 total drugs (13x6000).

XII. FURTHER RESEARCH

- Find and use a real dataset from lab testing. This would allow us to show the potential of our model and its usability within real-world lab testing environments.
- Speak and reach out to more scientists about our project and get their thoughts on it. Getting the perspective of someone who has made it their job to compare and contrast data from testing on biological subjects, especially with drugs, would help us gain a better understanding of

what we could do to improve our project, as well as make connections with those in the scientific world.

- Apply the model to a mobile net to make it more applicable to scientists, allowing them to use it in real-time. This would make our software easy to use and allow scientists to store data in the cloud, making the whole process quicker and more efficient.
- Add statistics software to our program to help visualize the differences between data. With statistics software automatically generating graphs from testing data, scientists could better compare and contrast the reactionary effects of different drugs more easily via a visual representation of the results.
- Allow scientists to input their observed features into an app to increase the range of our model. Our final and most ambitious goal would effectively strengthen our model by taking in input data from scientists who have tested it, expanding our training data, and improving the accuracy of our model.

XIII. ENDNOTE

The accuracy score that we ultimately achieved (100%) may not be ideal, however, as it may indicate that our model has been overfitted due to our testing process. We therefore would want to find and use existing datasets to modify and retest our algorithm.

REFERENCES

- [1] A Directed Acyclic Graph (DAG) Ensemble Classification Model: An Alternative Architecture for Hierarchical Classification Abstra, <https://core.ac.uk/download/pdf/82984976.pdf>. Accessed 7 June 2023.
- [2] Alam, Bashir, and Step Guide. "Decision Tree Python - Easy Tutorial 2023." Hands-On.Cloud, 12 January 2022, <https://hands-on.cloud/decision-tree-using-python/>. Accessed 7 June 2023.
- [3] Bryda, Elizabeth C. "The Mighty Mouse: The Impact of Rodents on Advances in Biomedical Research." NCBI, <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3987984/>. Accessed 7 June 2023.
- [4] "Estimated Research and Development Investment Needed to Bring a New Medicine to Market, 2009-2018." PubMed, 3 March 2020, <https://pubmed.ncbi.nlm.nih.gov/32125404/>. Accessed 7 June 2023.
- [5] Figueras, Albert. "Failures of the therapeutic chain as a cause of drug ineffectiveness : Promotion, misinformation, and economics work better than needs." NCBI, <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1125813/#B1>. Accessed 7 June 2023.
- [6] Guide, Step. "Confusion Matrix for Multi-Class Classification — Latest Guide 2023." Analytics Vidhya, 24 June 2021, <https://www.analyticsvidhya.com/blog/2021/06/confusion-matrix-for-multi-class-classification/>. Accessed 7 June 2023.
- [7] Guide, Step. "K-Nearest Neighbors (KNN) Classification with scikit-learn." DataCamp, <https://www.datacamp.com/tutorial/k-nearest-neighbor-classification-scikit-learn>. Accessed 7 June 2023.
- [8] Guide, Step, and Chirag Goyal. "Multiclass Classification Using SVM— SVM For Multiclass Classification." Analytics Vidhya, 18 May 2021, <https://www.analyticsvidhya.com/blog/2021/05/multiclass-classification-using-svm/>. Accessed 7 June 2023.
- [9] "How to successfully launch a rare disease drug in a patient-centric world." McKinsey, 9 January 2017, <https://www.mckinsey.com/industries/life-sciences/our-insights/how-to-successfully-launch-a-rare-disease-drug-in-a-patient-centric-world>. Accessed 7 June 2023.

- [10] Mondal, Pankaj. "The Importance of Pharmaceutical Competitor Analysis – DrugPatentWatch - Make Better Decisions." Drug-PatentWatch, <https://www.drugpatentwatch.com/blog/the-importance-of-pharmaceutical-competitor-analysis/>. Accessed 7 June 2023.
- [11] Normand, R., Du, W., Briller, M. et al. Found In Translation: a machine learning model for mouse-to-human inference. Nat Methods 15, 1067–1073 (2018). <https://doi.org/10.1038/s41592-018-0214-9>
- [12] "OvO and OvR Classifier." Michael Fuchs Python, 13 November 2019, <https://michael-fuchs-python.netlify.app/2019/11/13/ovo-and-ovr-classifier/>. Accessed 7 June 2023.
- [13] "Vaccine Research & Development." Johns Hopkins Coronavirus Resource Center, <https://coronavirus.jhu.edu/vaccines/timeline>. Accessed 7 June 2023.

APPENDIX A

Feature Explanation Document

Blood Pressure (mmHg):

What: Blood pressure is a measure of the force of blood against the walls of your arteries as it is pumped around your body by your heart. It is expressed as two numbers: systolic blood pressure (the higher number) and diastolic blood pressure (the lower number). The units of blood pressure are millimeters of mercury (mmHg).

Why: High blood pressure levels are major risks for both heart disease and strokes. Keeping track of this is greatly important due to the severe health risks that can ensue with consistently high blood pressure.

Levels: Low: 70-90 mmHg Good: 90-115 mmHg High: 115-130 mmHg

Heart Beats (BPM):

What: Heart rate is the measure of heart contractions within a period, this can help provide insight into the overall health of the animal.

Why: Heart rate offers insight into the overall health of an animal and is commonly used to quickly determine if something is wrong. Maintaining proper heart functions is crucial for any animal's survival, if any vaccine or drug changed these functions for the worse the animal's life may be at risk. By carefully monitoring heartbeats and abnormalities in the cardiovascular systems we can better determine the rats' overall health.

Levels: Low: 450-549 beats/min Good: 550-650 beats/min High: 651-750 beats/min

Weight Gain/Loss over 72 hours (% of body weight):

What: Sudden weight gain or loss could be caused by an under or overactive thyroid gland, which releases hormones that control metabolism. The rats could also be affected in other ways causing them to lose all appetite.

Why: Quickly gaining or losing weight in a short amount of time can be a symptom of a variety of problems within the body.

Levels:

Low: - 2-3% of total body weight **Good:** $\pm$ 0-1% of total body weight **High:** +2-3% of total body weight

Body Temperature (C):

What: Fever is one of the body's first reactions to infection because when the immune system notices a virus in your body, it alerts the hypothalamus to increase the body's temperature.

Why: Keeping track of internal temperatures can give insight into the rat's reaction to each drug. Changes in internal temperatures will allow us to determine what similar reactions may occur in humans after being injected with the trial drugs.

Levels: Low: 36-36.8 °C Good: 37.5-37.8 °C High: 38-39.8 °C

Cholesterol Levels (mg/dL):

What: Cholesterol is a waxy substance made by the liver and found in all cells of your body. Animal cells don't have cell walls like plant cells, so they use this substance to create a structurally integral foundation within the body.

Why: Having high levels of cholesterol can cause it to build up in your arteries, creating plaque that can stuff up and block blood flow. Low levels of cholesterol can increase the risk of having a hemorrhagic stroke, as well as other blood-related problems within the body.

Levels: Low: 45.18 ± 2.64 mg/dl Good: 49.64 ± 1.82 mg/dl High: 52 ± 1.54 mg/dl

Glomerular Filtration Rate (ml/min/100 g body weight)

What: Glomerular filtration rate (GFR) is a test used to determine how well the kidneys are functioning. Specifically, it estimates how much blood passes through the glomeruli each minute, glomeruli are tiny filters within the kidneys that filter waste from the blood.

Why: If our rats suffer from drastic changes in their kidney functions, kidney failure may be imminent and the rats may even die due to these failures.

Levels: Low: 0.5 ml/min/100g body weight Good: 1.00 ml/min/100g body weight High: 1.5 ml/min/100g body weight
Glucose Levels (mg/dL):

What: Glucose is a simple sugar that is necessary within our blood, as it provides us energy once broken down. Glucose can be attained through consuming foods.

Why: Your blood not only provides oxygen to the body but also glucose, with low or high levels of glucose permanent damage could ensue upon a multitude of important bodily functions such as the eyes, kidneys, and nerves. Monitoring this is important as any drastic and continual changes after receiving a drug would not be safe for use in humans.

Levels: Low: 30-50 mg/dL Good: 50-135 mg/dL High: 135-150 mg/dL

WBC (White Blood Cell Count):

What: WBC helps combat infections and viruses that threaten the rats' health. If a vaccine affected this balance and caused a change to this count, the rats' health would be at risk, therefore keeping track of WBC is an essential part of our testing.

Why: As our simulated rats will all have a similar mass, the number of white blood cells within their body will help determine the potency of the immune system, and how it may react against foreign substances that attack the body.

Levels: Low: $\leq 18,000$ Good: 18,000 - 22,000 High: $\geq 22,000$

Within the large category of WBC, there are many different subcategories or types of WBC which all play an important role in maintaining the health of the rats in our testing. To best track the health of the rats, keeping tabs on each category of WBC is essential. If one category strays low or high in value the rats' health could be at risk.

Neutrophils (% of WBC):

What:

Within the blood Neutrophils are the first responders, they destroy dangerous and infectious microorganisms by ingesting them and releasing enzymes that then kill them. The neutrophils also help boost the response of other white blood cells keeping infections and overall health risks at a minimum.

Why:

Neutrophils make up a majority of the total white blood cells in the body, so keeping healthy levels of neutrophils is, arguably, the most important as fewer other white blood cells are helping to make up for the losses. High levels of neutrophils can cause fevers and recurring infections, while low levels can cause extreme exhaustion and continual unending infections.

Levels: Low: 30-40% WBC Good: 40-60% of WBC High: 60-70% WBC Lymphocytes (% of WBC):

What:

Lymphocytes are produced within the bone marrow and have many important functions within the body such as producing antibodies and helping kill tumors and cancers. Additionally, lymphocytes help the immune system remember every antigen it comes in contact with, allowing the body to more efficiently fight it off during the next encounter.

Why:

Without proper levels of Lymphocytes, the body may be subject to the wrath of dangerous cancers. To help prevent this we keep track of the % composition of Lymphocytes in the blood after new vaccines have been injected.

Levels: Low: 15-20% WBC Good: 20-40% WBC High: 40-45% WBC

Monocytes (% of WBC):

What:

Like lymphocytes, monocytes are made within the bone marrow and mainly help find, and destroy germs and viruses that enter the bloodstream. Once they identify a dangerous invader they begin to eliminate it but also call for help from other white blood cells.

Why:

Ensuring the body can quickly identify viruses that pose a threat to the health of the animal is crucial. Monitoring monocytes, and ensuring they are kept at the appropriate level for healthy activity is highly important.

Levels: Low: 0.1-2% WBC Good: 2-8% WBC High: 8-10% WBC

Eosinophils (% of WBC):

What:

Eosinophils, like many of the other white blood cell types, are formed solely in the bone marrow. After fully maturing, the cells begin to fulfill their intended function of fighting off both infections and regulating the inflammatory process in the body. One of the special functions of eosinophils is responding to allergic reactions and disorders.

Why:

Eosinophiles' important role of moderating inflamed areas and fighting allergic disorders is essential in case of health risks. Without proper levels of eosinophils, the body is open and vulnerable to a multitude of life-threatening illnesses or diseases. High levels of eosinophils are telltale signs of parasitic infection or cancer. These major health risks are many reasons we keep track of eosinophils within the immune system.

Levels: Low: 0.1-1% WBC Good: 1-4% WBC High: 4-6% WBC

Basophils (% of WBC):

What:

Basophils are the least prominent type of white blood cells in the body, making up less than 1% of the total count, on top of this they have a very short lifespan of only 1-2 days. Regardless, basophils still play an important role in the immune system as they are the only type of white blood cells that can produce and release the enzyme histamine which helps fight against allergic reactions and asthma attacks.

Why:

Because basophils are the only cell type that can release histamine that inherently makes them important for the immune system. Having high levels of basophils is very dangerous and can be a sign of leukemia and autoimmune disease. Measuring levels will allow us to see any dangerous fluctuations after introducing vaccines.

Levels: Low: 0.1-0.5% WBC Good: 0.5-1% WBC High: 1-2% WBC

APPENDIX B

Drug Description Document

Drug A Blood Pressure: Low Heart Beats: Low Body Temperature: Low Cholesterol Level: Low Glomerular: Low Glucose Levels: Low Weight Gain/Loss: Low White Blood Cell Count: Low Neutrophils: Low Lymphocytes: Low Monocytes: Low Eosinophils: Low Basophils: Low

Drug B Blood Pressure: High Heart Beats: High Body Temperature: High Cholesterol Level: High Glomerular: High Glucose Levels: High Weight Gain/Loss: High White Blood Cell Count: High Neutrophils: High Lymphocytes: High Monocytes: High Eosinophils: High Basophils: High

Drug C Blood Pressure: High Heart Beats: Low Body Temperature: Low Cholesterol Level: High Glomerular: High Glucose Levels: Low Weight Gain/Loss: High White Blood Cell Count: Low Neutrophils: Low Lymphocytes: Low Monocytes: High Eosinophils: High Basophils: High Drug D Blood Pressure: Low Heart Beats: High Body Temperature: High Cholesterol Level: Low Glomerular: Low Glucose Levels: High Weight Gain/Loss: Low White Blood Cell Count: High Neutrophils: High Lymphocytes: High Monocytes: Low Eosinophils: Low Basophils: Low

Drug E Blood Pressure: Good Heart Beats: Good Body Temperature: Good Cholesterol Level: Good Glomerular: Good Glucose Levels: Good Weight Gain/Loss: Good White Blood Cell Count: Good Neutrophils: Good Lymphocytes: Good Monocytes: Good Eosinophils: Good Basophils: Good

Drug F Blood Pressure: High Heart Beats: High Body Temperature: Good Cholesterol Level: Low Glomerular: Good Glucose Levels: High Weight Gain/Loss: Low White Blood Cell Count: Low Neutrophils: High Lymphocytes: Good Monocytes: Good Eosinophils: Good Basophils: Good